



SigmaStar Camera 系统分区

© 2019 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.



REVISION HISTORY

Revision No.	Description	Date
{01}	{Initial release}	{07/05/2018}
{02}	{Refine}	{12/23/2019}

1. 目录

目录

REVISION HISTORY	ii
1. 目录.....	iii
2. 分区介绍	iv
2.1. SPI-NOR Flash 基本分区介绍:	iv
2.2. SPI-NAND Flash 基本分区介绍:	v
2.3. MISERVICE 分区内容:	vi
3. 分区变更:	vii
3.1. 分区变更前注意事项	vii
3.2. 分区变更的脚本代码架构介绍	vii
3.3. 分区位置变更	viii
3.3.1 Spinor flash	viii
3.3.2 Spinand flash	viii
3.4. 增加修改删除 jffs2 分区	ix
3.4.1 修改	ix
3.4.2 增加分区	ix
3.4.3 删除分区	ix
3.4.4 烧录脚本	ix
3.5. 增加修改删除 squashfs 分区	x
3.6. 增加修改删除 ubifs 分区	x
3.7. 特殊分区处理	x
4. MXP 与 PARTINFO	xi
4.1. SPINOR 的 MXP	xi
4.2. SPINAND 的 PARTINFO	xii
5. ONE BIN 介绍.....	xiv
5.1. Spinor	xiv
5.2. Spinand	xv

2. 分区介绍

2.1. SPI-NOR Flash 基本分区介绍:

ALKAID 在 make image 完成后会打印所有分区的 layout:

```
MXP count max 30
NOR FLASH HAS USED 0x1000000KB
  IPL: 0x00000000-0x00010000 size:64KB
  IPL_CUST: 0x00010000-0x00020000 size:64KB
  MXPT: 0x00020000-0x00030000 size:64KB
  UBOOT: 0x00030000-0x0004F000 size:124KB
  UBOOT_ENV: 0x0004F000-0x00050000 size:4KB
  BOOT: 0x00050000-0x00050000 size:320KB
  KERNEL: 0x00050000-0x00250000 size:2048KB
  rootfs: 0x00250000-0x00650000 size:4096KB
  miservice: 0x00650000-0x00950000 size:3072KB
  customer: 0x00950000-0x01000000 size:6848KB
Available MXP record count:10
```

IPL:

CPU 上电后首先跑到的是 rom code, 顾名思义代码保存在特殊的 ROM 中, 且是只读的。ROM code 跑完后会读取 NOR Flash 0 地址的位置, 这个位置就是 IPL 文件存放的位置, IPL 里主要功能是做一些基础的硬件初始化, 例如设定当前 DDR 参数, 以及 GPIO/IIC 相关等。

IPL_CUST:

IPL 初始化的是一些共有的硬件模块, IPL_CUST 中会根据当前板子的实际情况初始化客制化板子硬件的可执行的二进制文件, 例如客制化的 GPIO 管教, IIC 配置。

MXPT:

分区配置相关的二进制档案。

UBOOT:

UBOOT 的二进制文件存放分区。

UBOOT_ENV:

UBOOT 的环境变量存放分区。

LOGO:

在 NVR 设备上会使用, 存放的是开机 logo 相关的配置。

SPI-NOR 的 BOOT 分区中的内容是不建议改动的, 若有不符合公板 release 的地方, 可能会导致系统无法启动。

BOOT 的位置在 linux 在 mtd block0 的位置上, 对应的设备节点/dev/mtdblock0。

BOOT 分区之后的分区, 统称为 SYS 分区, SYS 分区是可以修改的, 根据实际的使用情况, 大致分为四个类别:

KERNEL:

存放内核的二进制文件。

ROOTFS:

如题。

miservice:

这是公板定义的一个分区, 它用来存储 mi 的库、一些配置文档, 文件系统默认 jffs2。

customer:

客户自己定制的分區。

2.2. SPI-NAND Flash 基本分区介绍:

Layout 图:

```

BOOT: 0x60000(CIS), 0x20000@0x60000(IPL0), 0x20000(IPL1), 0x20000(IPL2), 0x20000(I
SYS: 0x500000(KERNEL), 0x500000(RECOVERY), 0x600000(rootfs), -(UBI)
FLASH HAS USED 0x8000000KB
ChkSum      : 946
SpareByteCnt : 64
PageByteCnt  : 2048
BlkPageCnt   : 64
BlkCnt       : 1024
PartCnt      : 14
UnitByteCnt  : 8
Checksum ok!!
IDX:      StartBlk:      BlkCnt:      BackupBlkCnt:      PartType:
0:      0, (0000000000)    3, (0x00060000)    0, (0000000000)    0x0020, (CIS)
1:      3, (0x00060000)    1, (0x00020000)    0, (0000000000)    0x0003, (IPL)
2:      4, (0x00080000)    1, (0x00020000)    0, (0000000000)    0x0003, (IPL)
3:      5, (0x000A0000)    1, (0x00020000)    0, (0000000000)    0x0003, (IPL)
4:      6, (0x000C0000)    1, (0x00020000)    0, (0000000000)    0x0001, (IPL_CUST)
5:      7, (0x000E0000)    1, (0x00020000)    0, (0000000000)    0x0001, (IPL_CUST)
6:      8, (0x00100000)    1, (0x00020000)    0, (0000000000)    0x0001, (IPL_CUST)
7:      9, (0x00120000)    3, (0x00060000)    0, (0000000000)    0x0006, (UBOOT)
8:      12, (0x00180000)    3, (0x00060000)    0, (0000000000)    0x0006, (UBOOT)
9:      15, (0x001E0000)    1, (0x00020000)    0, (0000000000)    0x000D, (ENV)
10:     16, (0x00200000)    40, (0x00500000)    0, (0000000000)    0x0012, (KERNEL)
11:     56, (0x00700000)    40, (0x00500000)    0, (0000000000)    0x0009, (RECOVERY)
12:     96, (0x00C00000)    48, (0x00600000)    0, (0000000000)    0x000F, (ROOTFS)
13:    144, (0x01200000)    880, (0x06E00000)    0, (0000000000)    0x0021, (UBI)
[[ipl_nofsimag]]

```

CIS:

SPI-NAND 独有的分区，保存在 flash 0 地址的位置，它包含两部分内容，一部分是 spinand info，保存 spinand 的一些基本信息，例如 block page count、block count 等，这些信息会根据 spi nand 的种类而改变，另一部分是 partinfo，保存的分区信息，这些信息都是给 rom code 读取的，rom code 通过读取正确的 spinand 参数和分区信息，从而知道 IPL 的位置把整个系统跑起来。

IPL:

IPL 分区的作用与 SPI-NOR 分区一样，但是从上图可知，IPL 在 spi-nand 上保存了三份，目的是为了防止坏块而做的备份。

IPL_CUS:

作用同 SPI-NOR，同样会有三份。

UBOOT:

UBOOT 的二进制文件存放分区，有两份。

ENV:

UBOOT 的环境变量存放分区。

LOGO:

在 NVR 设备上会使用，存放的是开机 logo 相关的配置。

以上是 BOOT 相关的分区信息，这部分无法任意修改。

KERNEL:

存放内核的二进制文件。

ROOTFS:

如题。

UBI:

UBI 的内容在上图分区表中不会显示出来，UBI 中会创建多个 ubifs 格式的子分区，客户可以根据需要自行创建。Spinand 分区中的 miservice 分区就是放在 UBI 中。

2.3. MISERVICE 分区内容:

在 MISERVICE 分区的内容被 mount 到了根目录/config/下面，下表中加深字体的文件是必须保持路径一致的，否则有可能会系统无法启动。

└─ config	
└─ board.ini	——>保存当前板子的信息
└─ config_tool	——>mi sys 初始化 mmap 内存的应用
└─ dump_config -> config_tool	——>dump config 应用, config_tool 的 symbol link.
└─ dump_mmap -> config_tool	——>dump mmap 应用, config_tool 的 symbol link.
└─ fbdev.ini	——>Framebuffer 配置, 若无 fbdev 需求可删除。
└─ iqfile	
└─ imx307_iqfile.bin	
└─ iqfile0.bin -> imx307_iqfile.bin	——>Sensor IQ 相关的 bin
└─ iqfile1.bin -> imx307_iqfile.bin	
└─ iqfile2.bin -> imx307_iqfile.bin	
└─ iqfile3.bin -> imx307_iqfile.bin	
└─ isp_api.xml	
└─ mmap.ini	——>Mmap. Mmap 可以参考《Memory Layout 介绍.pdf》
└─ model	——>模块相关配置, 可以删除。
└─ Customer_1.ini	
└─ Customer_2.ini	
└─ Customer_auto_test.ini	
└─ modules	——>系统 KO 存放路径。
└─ 4.9.84	
└─ cifs.ko	
.....	
└─ pq	——>NVR 显示 PQ 相关。
└─ Bandwidth_RegTable.bin	
└─ Main.bin	
└─ Main_Ex.bin	
└─ terminfo	——>Console 显示相关。
└─ v	
└─ vt100	
.....	
└─ venc_fw	——>Encoder 的 firmware
└─ chagall.bin	

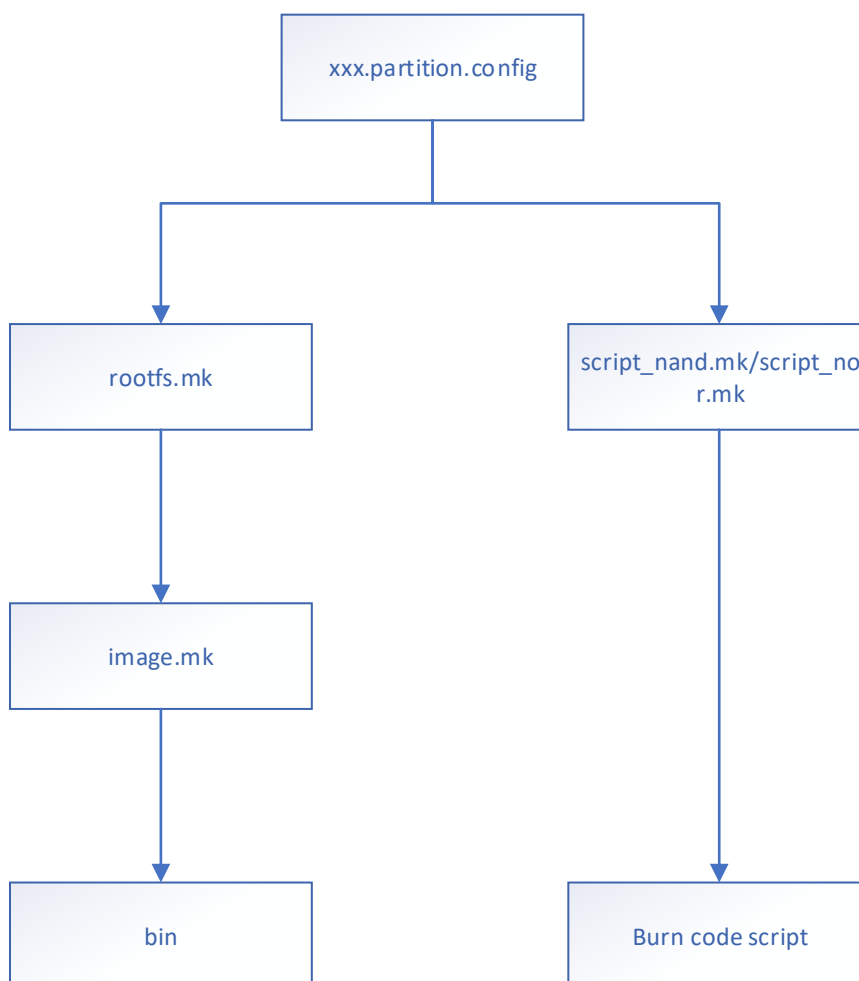
3. 分区变更：

3.1. 分区变更前注意事项

无论是 **spi-nor flash** 上的分区，还是 **spi-nand flash** 上的分区。都可归为两类，一类是不建议改动的 **boot**，一类是可以变动的 **sys**，这一章节介绍的分区变更都是在 **sys** 这部分上，包括如何配置脚本变更分区顺序，如何打包分区，如何生成烧录脚本，若客户有自己的一套打包和烧录的流程，可以不用关心这一章，需要特别注意的是在上一章中 **miservice** 分区中有特别标注出来的文件需要放到制定的路径下系统才能跑起来，所以在制作分区时务必在根目录下创建 **/config/**，并把相关的文件放到此路径下。

3.2. 分区变更的脚本代码架构介绍

分区变更分为分区制作、打包和分区烧录脚本制作这两部分，下面流程图的两个分支，左边走的是分区制作、打包流程，右边走的是分区烧录脚本制作的流程，分区烧录脚本仅支持 **tftp** 升级脚本，暂不支持 **usb** 升级脚本制作。



xxx.partition.config:

这是总的分区配置信息脚本，所有与分区调整相关的代码都在这个 **config** 文件中实现，这个 **config** 文件会有很多份，每个不同的 **chip**、**spinand** 分区还是 **spinor** 分区都会有各自的分区 **config** 文件。

通过 **build config**，就可以找到该项目使用的分区 **config** 文件是什么，例如 **SSC335** 的项目使用的 **build config**:

project/configs/ipc/i6b0/nor.glibc-squashfs.009a.64.qfn88

此配置脚本中，变量 **IMAGE_CONFIG** 可以找到对应使用的分区配置脚本。

此项目的分区配置脚本存放的路径在：**project/mage/configs/i6b0/nor.squashfs.partition.config**

可以看出，这是 **spinor flash** 分区配置脚本文件。

在分区 **config** 文件中用变量定义了分区 **BOOT** 的部分和 **SYS** 的部分：

Spinor:

mxp\$(BOOTTAB) = "xxx"

mxp\$(SYSTAB) = "xxx"

Spinand:

set_partition\$(BOOTTAB) = xxx

set_partition\$(SYSTAB) = xxx

在 **config** 脚本文件中，每个分区由一组变量表示，以上 **mxp** 和 **set_partition** 也是分区的一组变量之一，只不过这两个分区很特殊，存放的是一组规定的分区配置 **raw data**，这个是一个特殊的分区，特殊的分区由特别的指令生成，在后面章节会有介绍。

rootfs.mk:

这是 **ROOTFS** 和 **miservice** 分区制作脚本，**ssc335** 项目使用的是：**project/mage/configs/i6b0/rootfs.mk**

script_nand.mk/script_nor.mk:

这是制作 **spi-nand** 和 **spi-nor** 烧录脚本的脚本，**ssc335** 项目使用的是：

project/mage/configs/i6b0/script_nor.mk

project/mage/configs/i6b0/script_nand.mk

image.mk:

分区打包脚本，路径在 **project/mage/image.mk**。

3.3. 分区位置变更

3.3.1 Spinor flash

改变 **mxp\$(SYSTAB)**中所定义的分区前后位置即可，语法格式与 **linux uboot** 中使用的 **mtddpart** 是一样的，这个变量的值也会保存到 **uboot** 中的环境变量中。

mxp\$(SYSTAB) =

"\$(kernel\$(PATSIZE))(KERNEL),\$(rootfs\$(PATSIZE))(rootfs),\$(miservice\$(PATSIZE))(miservice),\$(customer\$(PATSIZE))(customer)"

其中 **kernel** 和 **rootfs** 的分区信息由逗号 **' , '** 隔开。

3.3.2 Spinand flash

改变 **set_partition\$(SYSTAB)**中所定义的分区前后位置即可，语法格式与 **linux uboot** 中使用的 **mtddpart** 是一样的，这个变量的值也会保存到 **uboot** 中的环境变量中。

set_partition\$(SYSTAB) = \$(kernel\$(MTDPART)),\$(rootfs\$(MTDPART)),-(UBI)

其中 **kernel** 和 **rootfs** 的分区信息由逗号 **' , '** 隔开。

3.4. 增加修改删除 jffs2 分区

在 spinor 的 flash 上一般用 jffs2 作为可读写的分区的文件系统，如下定义了 customer 分区的相关配置：

```
customer$(RESOUC)  = $(OUTPUTDIR)/customer
customer$(FSTYPE)   = jffs2
customer$(PATSIZE)  = 0x6B0000
customer$(MOUNTTG)  = /customer
customer$(MOUNTPT)  = mtd:customer
customer$(OPTIONS)  = rw
```

customer\$(RESOUC):

表示分区的资源文件放置的位置，rootfs.mk 和 image.mk 分别会往此路径中添加必须的文件，以及把文件夹打包成 bin 文件。

customer\$(FSTYPE):

打包的文件系统类型。

customer\$(PATSIZE):

分区大小。

customer\$(MOUNTTG)、customer\$(MOUNTPT)、customer\$(OPTIONS):

分区在板子起来后需要 mount 的路径以及参数。rootfs.mk 中会处理这些参数。

3.4.1 修改

若要修改分区，则改动以上变量值即可。

3.4.2 增加分区

1、假设增加一个分区名为 aaa，然后配置其大小等信息：

```
aaa$(RESOUC)  = $(OUTPUTDIR)/aaa
aaa $(FSTYPE)  = jffs2
aaa $(PATSIZE) = 0x6B0000
aaa $(MOUNTTG) = /aaa
aaa $(MOUNTPT) = mtd:aaa
aaa $(OPTIONS) = rw
```

2、分区相关的变量配置完成之后，在 mxp\$(SYSTAB) = "xxx"中指定其放置的位置。

3、在变量 "IMAGE_LIST" 后追加 "aaa"。

4、在表示需要 mount 的节点的变量 "USR_MOUNT_BLOCKS" 后追加 "aaa"。

3.4.3 删除分区

按照增加分区的操作，反过来执行。

3.4.4 烧录脚本

系统在 script_nor.mk 中会根据分区的配置自动生成。

3.5. 增加修改删除 squashfs 分区

Squashfs 的分区变更与 jffs2 分区变更大同小异，主要在 xxx\$(FSTYPE)上有不同，除去 rootfs 这个比较特别的分区之外，miservice 分区制作成 squashfs:

```
miservice$(RESOUCE) = $(OUTPUTDIR)/tvconfig/config
miservice$(FSTYPE) = squashfs
miservice$(PATSIZE) = 0x180000
miservice$(MOUNTTG) = /config
miservice$(MOUNTPT) = /dev/mtdblock3
miservice$(OPTIONS) = ro
```

需要特别注意的是，若使用者用的是 spinand 的分区配置脚本，那么分区位置放在：

set_partition\$(SYSTAB) = xxx 中。

与 jffs2 分区一样，系统在 script_nor.mk/script_nand.mk 中会根据分区的配置自动生成烧录脚本。

3.6. 增加修改删除 ubifs 分区

UBIFS 分区一般应用在 spinand 上作为可读写的分区文件系统。

UBIFS 的所有的分区都属于 UBI 的 mtd block 中的一个子分区。一个普通的 UBI 分区变量设置如下：

```
miservice$(RESOUCE) = $(OUTPUTDIR)/miservice/config
miservice$(FSTYPE) = ubifs
miservice$(PATSIZE) = 0xA00000
miservice$(MOUNTTG) = /config
miservice$(MOUNTPT) = ubi0:miservice
miservice$(OPTIONS) = rw
```

与 jffs2 分区变更方法不同的是，ubifs 分区不需要添加 mtdpart 讯息，也就是无需在 set_partition\$(SYSTAB) = xxx 中添加分区位置。

3.7. 特殊分区处理

KERNEL、uboot、IPL、logo 等分区属于特殊分区，这些分区没有特别的文件系统，所以无法在 Makefile 脚本中做集中处理，必须 by case 去处理。因此添加一个特殊的分区需要清楚了解如下两个步骤：

- 1、image.mk 中需要写上分区打包的脚本命令。

例如 IPL 分区打包命令：

```
ipl_nofsimage:
    @echo [[${@}]]
    cp -vf $(ipl$(RESOUCE)) $(IMAGEDIR)/boot/
```

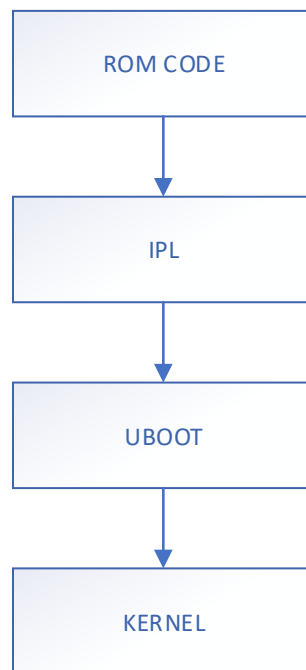
- 2、在 script_nor.mk/ script_nand.mk 中写上分区烧录脚本生成的命令。

例如制作 IPL 分区烧录脚本的脚本命令：

```
ipl_spi_nor_script:
    @echo "# <- this is for comment / total file size must be less than 4KB" >> $(SCRIPTDIR)/[[$(patsubst %_spi_nor__script,%,%)]]
    @echo mxx r.info IPL >> $(SCRIPTDIR)/[[$(patsubst %_spi_nor__script,%,%)]]
    @echo sf probe 0 >> $(SCRIPTDIR)/[[$(patsubst %_spi_nor__script,%,%)]]
    @echo sf erase \${sf_part_start} \${sf_part_size} >> $(SCRIPTDIR)/[[$(patsubst %_spi_nor__script,%,%)]]
    @echo tftp $(TFTPDOWNLOADADDR) boot/\$(ipl$(RESOUCE)) >> $(SCRIPTDIR)/[[$(patsubst %_spi_nor__script,%,%)]]
    @echo sf write $(TFTPDOWNLOADADDR) \${sf_part_start} \${sf_part_size} >> $(SCRIPTDIR)/[[$(patsubst %_spi_nor__script,%,%)]]
    @echo "% <- this is end of file symbol" >> $(SCRIPTDIR)/[[$(patsubst %_spi_nor__script,%,%)]]
```

4. MXP 与 PARTINFO

ARM 启动后会先进入一段固定的 ROM CODE 中：
如图所示：



ROM code 启动后会先去找一份原始的分区配置 raw data，这部分就是 MXP 或 PARTINFO。

MXP 与 PARTINFO 这两个分区分别保存的是 SPINOR/SPINAND 分区的配置信息，arm 启动执行到 ROM code/IPL 时由于在此时还没有 mtdpart 的概念，所以会去读 MXP 或 PARTINFO 中的内容才能找到 IPL 或者 uboot，因此这两个分区地址是固定的。

4.1. SPINOR 的 MXP

SPINOR 的分区修改全部汇总到 MXP_SF.bin，这个 bin 是由一个 tool 通过传入不同的参数动态生成。在 ALKAID 的编译打包环境中 image.mk 负责分区的打包步骤，通过 shell 脚本生成 MXP_SF.bin。

如同所示：

```

/home/mallic.peng/ALKAID/project/image/makefiletools/bin/mxpgenerator "0x10000(IPL),0x10000(IPL_CUST),0x10000(MXPT),0x1F000(UBOOT),0x1000(UBOOT_ENV)" "0x200000(KERNEL),0x400000(rootfs),0x600000(miservice),0x680000(customer)"
llloc.peng/ALKAID/project/image/output/images/boot/MXP_SF.bin
: [156]
BOOT: 0x10000(IPL),0x10000(IPL_CUST),0x10000(MXPT),0x1F000(UBOOT),0x1000(UBOOT_ENV)
SYS: 0x200000(KERNEL),0x400000(rootfs),0x600000(miservice),0x680000(customer)
MXP count max 30
NOR FLASH HAS USED 0x1000000KB
IPL: 0x00000000-0x00010000 size:64KB
IPL_CUST: 0x00010000-0x00020000 size:64KB
MXPT: 0x00020000-0x00030000 size:64KB
UBOOT: 0x00030000-0x0004F000 size:124KB
UBOOT_ENV: 0x0004F000-0x00050000 size:4KB
BOOT: 0x00000000-0x00050000 size:320KB
KERNEL: 0x00050000-0x00250000 size:2048KB
rootfs: 0x00250000-0x00650000 size:4096KB
miservice: 0x00650000-0x00950000 size:3072KB
customer: 0x00950000-0x01000000 size:6848KB
Available MXP record count:10
[[ip_nofsimage]]
  
```

工具 mxpgenerator: 执行命令:

```
mxpgenerator "0x10000(IPL),0x10000(IPL_CUST),0x10000(MXPT),0x1F000(UBOOT),0x1000(UBOOT_ENV)"
"0x200000(KERNEL),0x400000(rootfs),0x300000(miservice),0x6B0000(customer)"
/home/malloc.peng/ALKAID/project/image/output/images/boot/MXP_SF.bin
```

mxpgenerator 有三个参数 mxpgenerator [v0] [v1] [v2]:

v0 传入的是 BOOT 部分的分区配置, v1 传入的是 sys 分区的部分, v2 则是输出文件路径。

mxpgenerator 工具的源码路径:

project/image/makefiletools/src/mxpgenerator/mxpgenerator.c

利用服务器上的 gcc 编译:

gcc mxpgenerator.c -o mxpgenerator

4.2. SPINAND 的 PARTINFO

PARTINFO.pni 通过工具 pnigenerator 生成 PARTINFO.pni, 保存在 CIS 分区中, 其中包括一些 spinand 基本的讯息。

```
"/home/malloc.peng/ALKAID/project/board/i6b0/boot/spinand/partition/SPINANDINFO.sni" -> "/home/malloc.peng/ALKAID/project/image/output/images/boot/SPINANDINFO.sni"
/home/malloc.peng/ALKAID/project/image/makefiletools/bin/pnigenerator -s 64 -p 2048 -b 64 -k 1024 -u 8 -l 0x20000 -t "0x60000(CIS),0x20000@0x60000(IPL0),0x20000(IPL1),0x20000(IPL2),0x20000(IPL_CUST0),0x20000(IPL_CUST1),0x20000(IPL_CUST2),0x60000(UBOOT0),0x60000(UBOOT1),0x20000(ENV)" -y "0x500000(KERNEL),0x500000(RECOVERY),0x600000(rootfs),-(UBI)" -o /home/malloc.peng/ALKAID/project/image/output/images/boot/PARTINFO.pni
BOOT: 0x60000(CIS),0x20000@0x60000(IPL0),0x20000(IPL1),0x20000(IPL2),0x20000(IPL_CUST0),0x20000(IPL_CUST1),0x20000(IPL_CUST2),0x60000(UBOOT0),0x60000(UBOOT1),0x20000(ENV)
SYS: 0x500000(KERNEL),0x500000(RECOVERY),0x600000(rootfs),-(UBI)
FLASH HAS USED 0x8000000KB
chksum : 946
SpareByteCnt : 64
PageByteCnt : 2048
BlkPageCnt : 64
BlkCnt : 1024
PartCnt : 14
UnitByteCnt : 8
checksum ok!!
ID: StartBlk: BlkCnt: BackupBlkCnt: PartType:
0: 0,(0000000000) 3,(0x00060000) 0,(0000000000) 0x0020,(CIS)
1: 3,(0x00060000) 1,(0x00020000) 0,(0000000000) 0x0003,(IPL)
2: 4,(0x00080000) 1,(0x00020000) 0,(0000000000) 0x0003,(IPL)
3: 5,(0x000A0000) 1,(0x00020000) 0,(0000000000) 0x0003,(IPL)
4: 6,(0x000C0000) 1,(0x00020000) 0,(0000000000) 0x0001,(IPL_CUST)
5: 7,(0x000E0000) 1,(0x00020000) 0,(0000000000) 0x0001,(IPL_CUST)
6: 8,(0x00100000) 1,(0x00020000) 0,(0000000000) 0x0001,(IPL_CUST)
7: 9,(0x00120000) 3,(0x00060000) 0,(0000000000) 0x0006,(UBOOT)
8: 12,(0x00180000) 3,(0x00060000) 0,(0000000000) 0x0006,(UBOOT)
9: 15,(0x001E0000) 1,(0x00020000) 0,(0000000000) 0x000D,(ENV)
10: 16,(0x00200000) 40,(0x00500000) 0,(0000000000) 0x0012,(KERNEL)
11: 56,(0x00700000) 40,(0x00500000) 0,(0000000000) 0x0009,(RECOVERY)
12: 96,(0x00C00000) 48,(0x00600000) 0,(0000000000) 0x000F,(ROOTFS)
13: 144,(0x01200000) 880,(0x00600000) 0,(0000000000) 0x0021,(UBI)
[[ipl_nofsimage]]
cp -vf /home/malloc.peng/ALKAID/project/board/i6b0/boot/ipl/IPL.bin /home/malloc.peng/ALKAID/project/image/output/images/boot/
```

执行命令:

```
pnigenerator -s 64 -p 2048 -b 64 -k 1024 -u 8 -l 0x20000 -t
"0x60000(CIS),0x20000@0x60000(IPL0),0x20000(IPL1),0x20000(IPL2),0x20000(IPL_CUST0),0x20000(IPL_CUST1),0x20000(IPL_CUST2),0x60000(UBOOT0),0x60000(UBOOT1),0x20000(ENV)" -y
"0x500000(KERNEL),0x500000(RECOVERY),0x600000(rootfs),-(UBI)" -o
/home/malloc.peng/ALKAID/project/image/output/images/boot/PARTINFO.pni
```

pnigenerator [options]

参数含义:

- s Spare byte count.
- p Page byte count.
- b Block page count.
- k Block count.
- u Unit byte count.
- l Block size.



- t BOOT Part
- y SYS Part
- r Dump pni file
- o Output file path.

pnigenerator 工具的源码路径:

project/image/makefiletools/src/pnigenerator/pnigenerator.c

利用服务器上的 gcc 编译:

gcc pnigenerator.c -o pnigenerator

PARTINFO.bin 中保存 BOOT part 的分区信息就已经足够，其他的分区可以追加到 uboot 的 mtdparts 中。

5. ONE BIN 介绍

ONE BIN 功能是基于 ALKAID 的打包脚本通过 dd 命令使 BOOT PART 分区合并成一个 bin, 它可以支持 spinand 和 spinor 的两种分区打包方式。

在 project/image/boot.mk 中有 dd 合并 bin 的具体做法。

5.1. Spinor

参考脚本:

project/image/configs/i6b0/nor.squashfs.partition.boot.config

与普通的打包脚本对比:

1、增加了需要合并的分区名。

```
BOOT_IMAGE_LIST = ipl ipl_cust mxp uboot
```

2、合并的分区从 IMAGE_LIST 中剔除, 并把合并后的分区名加上

```
IMAGE_LIST = sstar kernel rootfs miservice customer
```

3、添加 sstar 分区。

```
sstar$(RESOURCE) = $(IMAGEDIR)/boot/sstar.bin
```

```
sstar$(PATSIZE) = $(shell printf 0x%x $(shell stat -c "%s" $(sstar$(RESOURCE))))
```

4、同时在 image.mk 和 script_nor.mk 中需要添加 sstar 的 image 生成脚本和 tftp 烧录脚本:

image.mk:

```
sstar_nofsimage: sstar_dd

ifeq ($(FLASH_TYPE), nor)
sstar_dd: mxp_nofsimage
    @echo [[${@}]]
    dd if=/dev/zero of=$(sstar$(RESOURCE)) bs=1 count=0
else
sstar_dd:
    @echo [[${@}]]
    dd if=/dev/zero of=$(sstar$(RESOURCE)) bs=1 count=0
endif
```

script_nor.mk:

```
sstar_spi_nor_script:
    @echo "# <- this is for comment / total file size must be less than 4KB" > $(SCRIPTDIR)/[[$(patsubst %_spi_nor__script,%,%)]]
    @echo sf probe 0 >> $(SCRIPTDIR)/[[$(patsubst %_spi_nor__script,%,%)]]
    @echo tftp $(TFTPDOWNLOADADDR) boot/sstar.bin >> $(SCRIPTDIR)/[[$(patsubst %_spi_nor__script,%,%)]]
    @echo sf erase 0 \${${filesize}} >> $(SCRIPTDIR)/[[$(patsubst %_spi_nor__script,%,%)]]
    @echo sf write $(TFTPDOWNLOADADDR) 0 \${${filesize}} >> $(SCRIPTDIR)/[[$(patsubst %_spi_nor__script,%,%)]]
    @echo mxp t.load >> $(SCRIPTDIR)/[[$(patsubst %_spi_nor__script,%,%)]]
    @echo "% <- this is end of file symbol" >> $(SCRIPTDIR)/[[$(patsubst %_spi_nor__script,%,%)]]
```

5.2. Spinand

由于 CIS 分区的特殊性，CIS 分区中的 pni 和 sni 无法做成 one bin 包，因此 spinand 仅仅把 ipl/uboot 进行了合并。

参考脚本：

project/image/configs/i6b0/spinand.squashfs.partition.boot.config

与普通的打包脚本对比：

- 1、增加了需要合并的分区名。

```
BOOT_IMAGE_LIST = ipl ipl_cust uboot
```

- 2、合并的分区从 IMAGE_LIST 中剔除，并把合并后的分区名加上

```
IMAGE_LIST = set_partition sstar kernel rootfs miservice customer
```

- 3、MTDPART 设定更新：

```
MTDPARTS = "mtdparts=nand0:$(sstar$(MTDPART)),$(set_partition$(SYSTAB))"
```

- 4、添加 sstar 分区。

```
sstar$(RESOUC) = $(IMAGEDIR)/boot/sstar.bin
```

```
sstar$(PATSIZE) = $(shell printf 0x%x $(shell stat -c "%s" $(sstar$(RESOUC))))
```

```
sstar$(MTDPART) = $(sstar$(PATSIZE))@$(set_partition$(CISSIZE))(BOOT),0x20000(ENV)
```

- 5、由于 IPL/UBOOT 分区有多个备份，所以要指定备份数量：

```
ipl$(RESOUC) = $(PROJ_ROOT)/board/$(CHIP)/boot/ipl/IPL.bin
```

```
ipl$(PATSIZE) = 0x20000
```

```
ipl$(COPYES) = 3
```

```
ipl$(MTDPART) =
```

```
$(ipl$(PATSIZE))@$(set_partition$(CISSIZE))(IPL0),$(ipl$(PATSIZE))(IPL1),$(ipl$(PATSIZE))(IPL2)
```

- 6、同时在 image.mk 和 script_nand.mk 中需要添加 sstar 的 image 生成脚本和 tftp 烧录脚本：

image.mk:

```
sstar_nofsimage: sstar_dd

ifeq ($(FLASH_TYPE), nor)
sstar_dd: mxp_nofsimage
    @echo [[${@}]]
    dd if=/dev/zero of=$(sstar$(RESOUC)) bs=1 count=0
else
sstar_dd:
    @echo [[${@}]]
    dd if=/dev/zero of=$(sstar$(RESOUC)) bs=1 count=0
endif
```

script_nand.mk:

```
sstar_$(FLASH_TYPE)__script:
@echo "# <- this is for comment / total file size must be less than 4KB" > $(SCRIPTDIR)/[[sstar.es
@echo tftp $(TFTPDOWNLOADADDR) boot/sstar.bin >> $(SCRIPTDIR)/[[sstar.es
@echo nand erase.part BOOT >> $(SCRIPTDIR)/[[sstar.es
@echo nand write.e $(TFTPDOWNLOADADDR) BOOT \${filesize} >> $(SCRIPTDIR)/[[sstar.es
@echo "% <- this is end of file symbol" >> $(SCRIPTDIR)/[[sstar.es
```